

Hyperparameter Sensitivity in Cooperative Multi-Agent Reinforcement Learning: A MultiWalker Study

Ruzhica Chochoroska 216057

<https://github.com/ruzicaa06/cooperative-multiwalker-marl>

Abstract

Cooperative multi-agent reinforcement learning (MARL) presents significant challenges due to non-stationarity and sensitivity to hyperparameters. In this project, the performance and training stability of PPO, DDPG and SAC were evaluated in the MultiWalker environment, a continuous-control task. The results show that standard PPO and DDPG suffer from high variance and frequent coordination failures, while SAC achieves modest stability with limited cooperative performance. By applying automated hyperparameter optimization with Optuna, PPO achieves substantially higher mean rewards with reduced variability, demonstrating the critical role of hyperparameter tuning in cooperative MARL. Robust cooperative MARL requires both algorithm choice and systematic hyperparameter tuning.

1 Introduction

Cooperative MARL involves multiple agents learning to act in a shared environment toward a common objective, requiring coordination for maximizing shared rewards. Applications include multi-robot systems, swarm robotics, autonomous driving and distributed control. A central challenge is non-stationarity: from an agent’s perspective, the environment changes as other agents update policies, often causing instability, high variance and coordination collapse.

Applying single-agent RL algorithms to joint observation-action spaces under centralized training is a common baseline but remains sensitive to hyperparameters, which can determine whether coordination emerges or fails.

This project investigates hyperparameter sensitivity in cooperative MARL using MultiWalker, a continuous-control task demanding precise coordination. It evaluates PPO, DDPG and SAC under centralized training and applies automated hyperparameter optimization with Optuna for PPO.

2 Related Work

Cooperative MARL faces challenges from non-stationarity, particularly in continuous-control domains, leading to unstable training and coordination failures [1]. Common benchmarks include MultiWalker, Cooperative Navigation and Multi-Agent MuJoCo [2, 3]. MultiWalker requires tight synchronization, making performance sensitive to policy instability and reward variance [4].

On-policy methods like PPO are popular due to relative stability [5]. Centralized training with parameter sharing improves reliability but remains hyperparameter-sensitive [6]. Off-policy algorithms such as DDPG and SAC offer better sample efficiency but suffer higher variance and instability in cooperative tasks [3, 7, 4].

Automated hyperparameter optimization has improved single-agent RL [8] and tools like Optuna efficiently search hyperparameter spaces [9]. This project empirically examines hyperparameter sensitivity in cooperative MARL, analyzing its effect on stability, reward variance, non-monotonic dynamics and coordination collapse in MultiWalker using PPO, DDPG and SAC and quantifies gains from automated PPO tuning.

3 Methodology

3.1 Environment and Setup

Experiments were conducted in the *MultiWalker* environment from the PettingZoo SISL suite [10]. MultiWalker is a continuous-control cooperative task where multiple bipedal agents must coordinate to transport a shared payload across uneven terrain. Episodes terminate early if any agent falls or the payload is dropped and are capped at 500 simulation steps.

The `multiwalker_v9` parallel API was used with three walkers. Each agent observes local joint states, contact information and the relative position of the payload and outputs continuous joint-control actions. All agents receive an identical shared reward encouraging forward progress while penalizing instability. Small stochastic perturbations were applied to joint positions and angles to improve robustness.

To ensure compatibility with Stable-Baselines3, the PettingZoo environment was wrapped using SuperSuit [11] and converted into a vectorized interface. A centralized training formulation was used, treating the multi-agent system as a single joint decision-making process, which is a common baseline for cooperative MARL with shared rewards.

3.2 Algorithms and Training Configuration

Three reinforcement learning algorithms from Stable-Baselines3 [12] were evaluated: Proximal Policy Optimization (PPO), Deep Deterministic Policy Gradient (DDPG) and Soft Actor-Critic (SAC). All models were trained for 2 million environment timesteps under identical environment and evaluation settings.

PPO [5] was trained using rollouts of 2048 steps, batch size 64 and 10 optimization epochs per update. The discount factor was $\gamma = 0.99$ and Generalized Advantage Estimation used $\lambda = 0.95$. The learning rate was 3×10^{-4} , clipping range 0.2, entropy coefficient 0.01 and gradient norm clipping 0.5.

DDPG [13] was trained with a replay buffer of size 10^6 and minibatch size 64. Actor and critic networks used two hidden layers with 256 units. The learning rate was 10^{-3} , $\gamma = 0.99$ and Polyak averaging with $\tau = 0.005$ was used for target updates.

SAC [14] was trained with a replay buffer of size 10^6 and batch size 256. Actor and critic networks used two hidden layers with 256 units. The learning rate was 3×10^{-4} , $\gamma = 0.99$ and Polyak averaging with $\tau = 0.005$ was used. The entropy coefficient was learned automatically.

All algorithms used multilayer perceptron policies with ReLU activations. Parameters were shared across agents. Optimization used Adam and gradient clipping was applied to improve stability.

3.3 Evaluation Protocol

Policies were evaluated using deterministic action selection without exploration noise. Mean episode return was computed across agents. Evaluations were conducted every 10,000 timesteps over 10 episodes using a separate evaluation environment and the checkpoint with the highest mean evaluation reward was retained. Learning curves and final metrics were averaged across random seeds where applicable.

3.4 Hyperparameter Optimization for PPO

Baseline PPO showed high variance and unstable coordination, indicating strong hyperparameter sensitivity. PPO hyperparameters were optimized using Optuna [9] by maximizing mean evaluation reward. A Tree-structured Parzen Estimator sampler was used and a Median Pruner terminated underperforming trials early.

Each Optuna trial trained PPO for 300,000 timesteps with evaluation every 10,000 steps over 5 episodes. The best configuration from 60 trials was retrained for 2 million timesteps under the same protocol as the baseline PPO.

4 Results

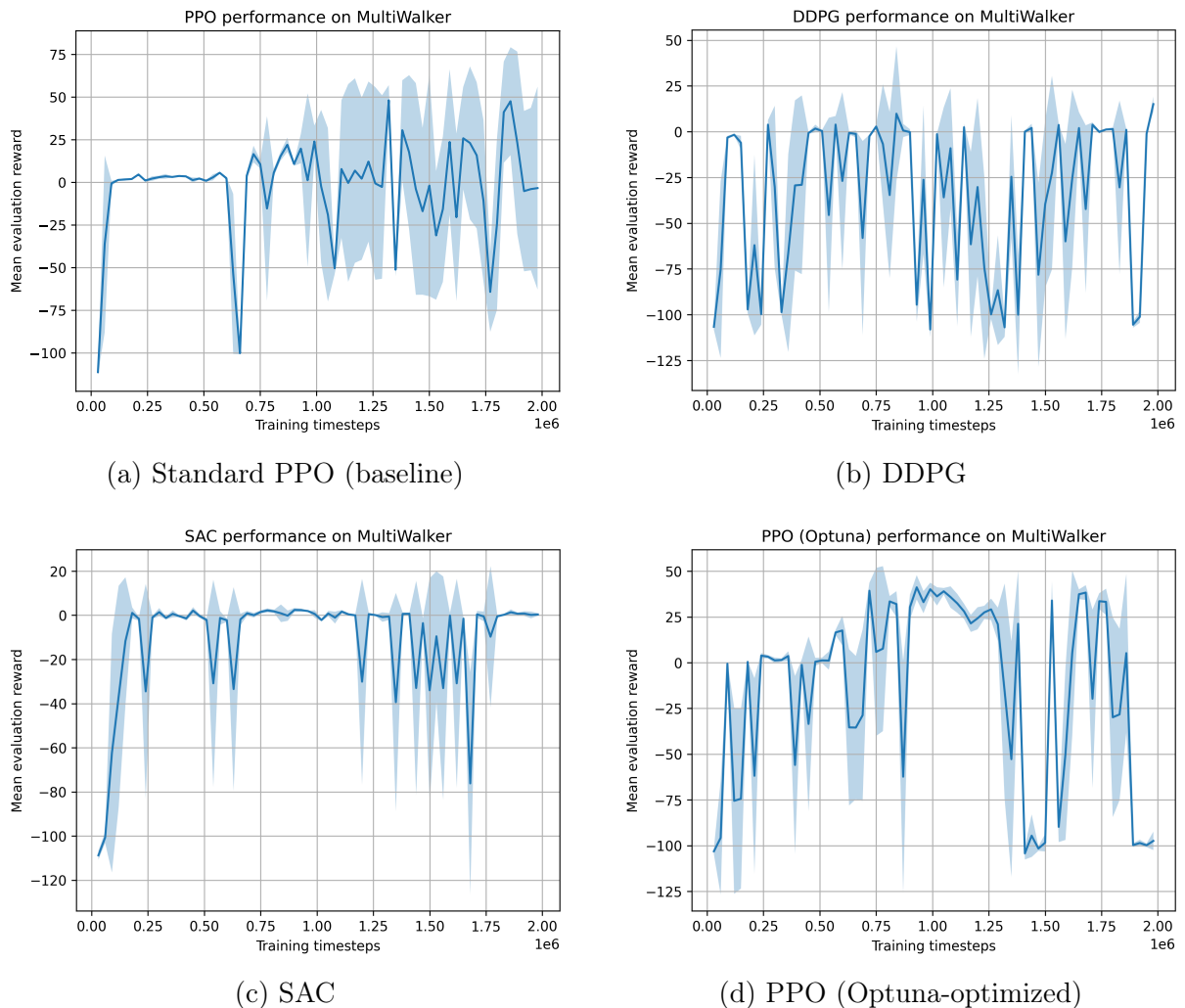


Figure 1: Training dynamics of the evaluated algorithms in the MultiWalker environment: mean evaluation reward ± 1 standard deviation.

4.1 Training Dynamics

Figures 1a, 1b, 1c and 1d show training curves for PPO, DDPG, SAC and Optuna-optimized PPO in MultiWalker. Curves report mean evaluation reward with shaded regions indicating ± 1 standard deviation. Evaluations were performed periodically using deterministic policies and averaged over multiple episodes.

4.1.1 PPO

Baseline PPO shows unstable learning with high variance. Early training yields strongly negative rewards (e.g., -111 at 30k and -36 at 60k), reflecting frequent failures. Performance gradually improves toward zero by 300k timesteps (approximately 3.0), suggesting partial emergence of coordination. However, training remains highly non-monotonic, with repeated collapses after temporary improvements. Peak rewards exceed 20, with a maximum observed mean reward of approximately 48 around 1.32M timesteps. The best PPO checkpoint achieves a mean reward of 4.98 with a standard deviation of ± 50.20 over 100 evaluation episodes (Table 1).

4.1.2 DDPG

DDPG performs poorly throughout training, with rewards remaining below -90 up to approximately 180k timesteps and consistently high variance. Short-lived improvements occur (e.g., mean reward ≈ 3.8 at 270k), but are repeatedly followed by collapses. The best DDPG model achieves a mean reward of -18.76 with a standard deviation of ± 52.11 , indicating failure to learn stable cooperative behavior.

4.1.3 SAC

SAC improves earlier than DDPG and shows relatively stable behavior, with rewards approaching zero by around 180k timesteps. From 180k onward, performance remains within a narrow band near zero with lower variance than PPO and DDPG and collapses are typically followed by recovery. However, SAC rarely achieves strong cooperative performance, with maximum mean reward remaining below 3.0. The best SAC checkpoint achieves a mean reward of -4.18 with a standard deviation of ± 23.40 .

4.1.4 Optuna-Optimized PPO

Optuna-optimized PPO demonstrates substantially improved cooperative performance but retains non-monotonic dynamics. Early training remains negative (e.g., -103.13 at 30k), reaching near-zero performance by 90k timesteps (-0.58). Between 90k and 360k, performance fluctuates with high variance, including brief positive phases (e.g., 3.86 at 240k) followed by regressions. A strong improvement phase occurs between 570k and 1.08M timesteps, where mean rewards frequently exceed 30. The best performance is achieved at 930k timesteps with a mean reward of 41.21 and standard deviation ± 6.63 . Later training again exhibits collapses followed by recovery, indicating that the best policy occurs at an intermediate checkpoint rather than the final model.

4.2 Cross-Algorithm Comparison

Table 1 summarizes the best-performing models over 100 evaluation episodes. PPO and DDPG exhibit high variance and frequent catastrophic failures, while SAC is more stable but achieves limited cooperative reward. In contrast, Optuna-optimized PPO achieves substantially higher mean reward (41.21) with significantly reduced variance (± 6.63), demonstrating that hyperparameter tuning is critical for achieving stable coordination in cooperative MARL.

Table 1: Performance of best models across algorithms in the MultiWalker environment.

Algorithm	Mean Episode Reward	Std. Dev.
PPO	4.98	± 50.20
DDPG	-18.76	± 52.11
SAC	-4.18	± 23.40
PPO (Optuna)	41.21	± 6.63

4.3 Videos

Qualitative results showing agent coordination and behavior for all algorithms are available in the following folder: <https://github.com/ruzicaa06/cooperative-multiwalker-marl/tree/main/videos>.

5 Conclusion

In this project, the performance and training stability of PPO, DDPG and SAC were evaluated in the cooperative MultiWalker environment, with a focus on hyperparameter sensitivity in multi-agent reinforcement learning. The results highlight the inherent difficulty of learning stable coordinated behaviors when single-agent reinforcement learning algorithms are applied to cooperative multi-agent tasks under centralized training.

Standard PPO demonstrated the ability to discover effective cooperative strategies but suffered from high variance and frequent catastrophic failures throughout training. DDPG exhibited the poorest performance, with persistent instability and repeated collapses to degenerate policies, indicating strong sensitivity to non-stationarity introduced by experience replay. SAC achieved improved robustness and lower variance compared to PPO and DDPG, but its performance remained limited, rarely converging to highly coordinated solutions.

A key outcome of this project is the significant impact of hyperparameter optimization on cooperative learning performance. When PPO was optimized using Optuna, a substantial improvement was observed, achieving a high mean episode reward of 41.21 with low variability (± 6.63). Despite this improvement, training dynamics remained highly non-monotonic, with abrupt transitions between coordinated and uncoordinated behaviors. This emphasizes the importance of selecting models based on evaluation performance rather than final training checkpoints.

Overall, the findings demonstrate that algorithm choice alone is insufficient to ensure reliable learning in cooperative multi-agent environments. Instead, systematic hyperparameter optimization is essential and can determine whether effective coordination is achieved. These results reinforce hyperparameter sensitivity as a central challenge in cooperative multi-agent reinforcement learning and motivate the use of automated optimization techniques for robust policy discovery.

6 Limitations and Future Work

This project has several limitations that suggest directions for future research. First, all experiments were conducted using centralized training with single-agent algorithm implementations applied to joint observation and action spaces. While this provides a strong baseline for comparison, it does not address decentralized execution or scalability, which are critical considerations in real-world multi-agent systems.

Second, automated hyperparameter optimization was applied only to PPO due to computational constraints. Extending optimization to off-policy algorithms such as SAC and DDPG may improve their stability and performance and enable a more comprehensive comparison. In addition, the Optuna objective emphasized short-horizon evaluation performance, which may favor hyperparameters that perform well early but degrade during longer training.

Future work may explore decentralized or parameter-sharing architectures, explicit inter-agent communication or multi-agent-specific algorithms such as MAPPO or QMIX. Further improvements may also be achieved through curriculum learning, adaptive hyperparameters or population-based training. Finally, evaluating performance across different numbers of agents and environment configurations would provide additional insight into generalization and scalability in cooperative multi-agent reinforcement learning.

References

- [1] Pablo Hernandez-Leal, Bilal Kartal, and Matthew E. Taylor. A survey and critique of multiagent deep reinforcement learning with applications to cooperative problems. *arXiv preprint arXiv:1811.12560*, 2019.
- [2] Jayesh K Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *International conference on autonomous agents and multiagent systems*, pages 66–83. Springer, 2017.
- [3] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative, multi-agent games. *arXiv preprint arXiv:2103.01955*, 2021. Also appeared at NeurIPS 2022 Datasets and Benchmarks Track.
- [5] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [6] Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. What matters for on-policy deep actor-critic methods? a large-scale study. In *International conference on learning representations*, 2021.
- [7] Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms on cooperative tasks. *arXiv preprint arXiv:2006.07869*, 2020.
- [8] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- [9] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.

- [10] Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34:15032–15043, 2021.
- [11] J. K. Terry, Benjamin Black, and Ananth Hari. Supersuit: Simple microwrappers for reinforcement learning environments. *arXiv preprint arXiv:2008.08932*, 2020. doi: 10.48550/arXiv.2008.08932. URL <https://arxiv.org/abs/2008.08932>.
- [12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.
- [13] Timothy P. Lillicrap et al. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [14] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.